project paper

This project has implemented an LLM-based automatic white-box test case generation tool. Operating as a Python CLI, it supports two target languages (Python/JavaScript) and is compatible with multiple LLM backends including DeepSeek, Google Gemini, and GitHub GPT-4o-mini. Its core functionalities include code analysis, test case generation, coverage measurement, closed-loop supplementation, and multi-LLM comparison. This report will analyze three aspects: comparison with traditional non-AI white-box testing techniques, AI limitations encountered in practice and corresponding tool improvement methods, and project summary with future outlook.

1、Comparison with Traditional Non-AI White-box Testing Technologies

1.1 Advantages and Limitations of Traditional Technologies

Traditional non-AI white-box testing techniques mainly include AST-based static generation tools, symbolic execution, fuzzing, and manual test design. Each approach has its focus, yet all exhibit significant limitations in practical application.

AST-based static generation tools (e.g., JUnit Generator, PyTest Generator) can automatically generate test method skeletons based on code structure but only produce "empty shells"—test methods are created, while specific input data and assertions still require manual specification by testers. Taking the target module `convert_number_to_words.py` in this project as an example, such tools can generate a test method named `test_convert_number` but cannot infer whether the expected output for input `123` should be "one hundred twenty-three" or "一百二十三". Testers must still understand the business logic and manually complete the assertions, resulting in limited automation.

Symbolic execution techniques (e.g., KLEE) generate test cases covering extreme paths by solving path constraints, offering strong theoretical coverage capabilities. However, in practice, when faced with recursive functions such as `convert_number`, the number of paths grows exponentially with the number of input digits, easily leading to "path explosion" that exhausts memory or renders computation time unacceptable. For more complex libraries like Dayjs, symbolic execution struggles to handle internal date calculations and timezone conversions.

Fuzzing (e.g., AFL) excels in security testing by generating random inputs and monitoring for program anomalies. However, its objective is to discover crashes and memory leaks rather than achieving high statement or branch coverage. For convert_number, fuzzing might generate -9999999 and verify that the program does not crash, but it will not check whether the output is "negative nine million nine hundred ninety-nine thousand nine hundred ninety-nine", thus failing to guarantee functional correctness.

Manual test case design is the most reliable approach. In this project, we designed 17 test cases for convert_number based on requirement documentation, covering all independent paths with 100% accuracy. However, this approach incurs extremely high costs—designing test cases for a mere 52-line function required approximately one hour, while testing a 500+ line library like Dayjs would require several days. In the context of rapid iteration in modern software development, such substantial human investment is unsustainable.

1.2 Core Advantages of This Project

The LLM-based test generation tool effectively complements the aforementioned traditional approaches, with its core advantages manifested in the following aspects:

Dimension	Traditional Technique Pain Points	Response to this project
Assertive Filling	AST tools generate only test skeletons; assertions require manual filling	LLM directly generates complete input-output assertions, e.g., expect(convert_number(123)).toBe("one hundred twenty-three")
Path Coverage	Symbolic execution prone to path explosion	LLM combined with AST analysis achieves 91% statement coverage and 92.3% branch coverage in the first round
Business Semantics	Fuzzing cannot verify output correctness	LLM can incorporate requirement documentation to generate assertions conforming to business rules
Efficiency	Manual design is time-consuming	Generation, execution, and closed-loop supplementation of 24 test cases total only 138 seconds

- **Multi-mode Adaptability**: To address different code styles, this project designed two generation modes. For pure functional code like convert_number, the JSON test case mode is employed: the LLM outputs a JSON structure such as [{"input":123,"expected":"one hundred twenty-three"}], which the tool converts to pytest assertions. For chain-call libraries like Dayjs, the JSON mode completely fails—dayjs('2023-01-01').format('YYYY') cannot be expressed as {input:"2023-01-01", expected:"2023"}. Therefore, we introduced the --script mode, allowing the LLM to directly generate executable test script code, successfully increasing Dayjs statement coverage from 26.9% to 97.1%.
- **AST Integration for Enhanced Understanding**: Simply sending source code text to the LLM may cause the model to overlook hidden branches. Taking the match-case structure in the max_value function as an example, plain text cannot clearly convey the specific logic of each branch. By extracting structured branch information through AST—SHORT: max_exp = value[0][0] + 3, LONG: max_exp = value[0][0] + 6, INDIAN: max_exp = 19, _: raise ValueError — and injecting it into the prompt, the LLM-generated test cases precisely cover all four paths.
- **Closed-loop Feedback Mechanism**: Traditional techniques generate test cases in a "one-shot" manner; if coverage targets are not met, manual supplementation is required. This project implements an automated closed loop: run tests →collect uncovered lines and branches →send uncovered code fragments to the LLM →generate supplementary test cases. For Python projects, first-round coverage reaches over 90%; for Dayjs projects, after 2-3 rounds of closed-loop iteration, coverage improves from approximately 30% to between 70% and 97%.

1.3 Limitations of this project

Despite its numerous advantages, this project is subject to constraints inherent to LLMs, resulting in the following limitations:

- **LLM Hallucination**: In the Dayjs project, some models generated assertions such as expect(...).toBe(1.5), while the actual computed result was 1.5645.... The LLM assumed,

based on common sense, that the month difference should be a neat 1.5, ignoring complex factors such as leap years and varying month lengths. Such hallucinations are less frequent in pure functional projects but are difficult to eliminate in scenarios involving dates, floating-point operations, and similar complexities.

- **Result Randomness**: Running the same model on the same code multiple times may yield different numbers of generated test cases and varying accuracy rates. For instance, DeepSeek Reasoner's accuracy on convert_number ranged from 86.96% to 95.00% across multiple experiments. Traditional techniques do not exhibit such uncertainty.
- **Significant Performance Variance**: Response times across different LLMs vary dramatically. GPT-4o-mini generates Python test cases in just 32.7 seconds, while DeepSeek Reasoner requires 271.5 seconds—a difference exceeding eightfold. For the Dayjs project, Reasoner's runtime reached 670 seconds, severely impacting iteration efficiency.
- **Closed-loop Overfitting**: During the supplementary generation phase, LLMs sometimes become overly focused on covering a single uncovered line, generating extreme or even invalid inputs that cause assertion failures. For example, to cover the tens == 1 branch in convert_small_number, the LLM generated num=100. However, this input is intercepted at the function entry by if num >= 100 and raises an exception, never reaching the target branch.

2. AI Limitations Encountered in Practice and Improvement Process

Throughout the project development and experimentation process, we encountered a series of practical problems stemming from LLM characteristics. These issues span multiple dimensions, including code structure understanding, output format stability, and contextual integrity. The following sections detail the specific AI limitations we encountered and the corresponding improvement processes.

2.1 Expressiveness Limitations of the JSON Format: LLMs Writing Test Scripts Directly

In the early stages of the project, we designed a JSON-based test case format, expecting to

uniformly describe all testing scenarios using the "input-output" paradigm. While this design performed well for pure functional Python code, it exposed fundamental deficiencies when applied to chain-call libraries such as Dayjs. A typical Dayjs testing scenario requires expression as:

```
const result = dayjs('2023-01-01').add(1, 'month').format('YYYY-MM-DD');
expect(result).toBe('2023-02-01');
```

However, the JSON format can only express {"input": "2023-01-01", "expected": "2023-02-01"}, completely losing the intermediate method call chain. The essence of the problem lies in the fact that chain calls are not suited to the "input→output" abstraction, and JSON as an intermediate format has inherent expressiveness limitations.

Therefore, we re-evaluated the tool's design abstraction and decided to abandon the intermediate format, instead having the LLM directly output executable test scripts. This led to the development of the --script mode, incorporating three key design elements:

- **Initial Generation**: The prompt includes import statements and basic structure examples for the testing framework, requiring the LLM to output complete test file code.
- **Closed-loop Supplementation**: To avoid truncation risks associated with full script regeneration, we adopted a "fragment concatenation" strategy—the LLM generates only new test method fragments, which the program automatically appends before the closing }); of the existing script. Concurrently, the prompt sends only uncovered lines and their surrounding three lines of context, significantly reducing input length.
- **Quality Assurance**: Immediately after concatenation, syntax validation is performed (node --check for JavaScript). If syntax errors are detected, the supplement is discarded and the previous version retained. A rollback mechanism ensures the highest-coverage script version is always preserved.

After these improvements, DeepSeek Chat's statement coverage increased from 26.9% to 72.7%, while DeepSeek Reasoner reached 97.1%; accuracy improved from 0% to 97.37%.

2.2 Coverage Gaps in AST Analysis: The Analysis Module Requires Continuous Evolution

While analyzing the max_value function in convert_number_to_words.py, we discovered that the AST analysis results showed an empty "branch points" list, despite the actual code containing a match-case structure with four distinct branch paths. Investigation revealed that the PythonAnalyzer module initially only handled four statement types—if, for, while, and try—and completely ignored the match statement introduced in Python 3.10.

This occurred because the Python language encompasses dozens of statement types, while JavaScript has even more. Our initial assumption—that core control flow consists only of if, for, and while statements — overlooked the reality of language evolution. With the widespread adoption of Python 3.10, match-case structures are increasingly used, and such omissions result in incomplete code structure information provided to the LLM, adversely affecting test case generation quality.

The corresponding solution was to extend the PythonAnalyzer with a visit_Match method that traverses node.cases to extract each branch's pattern and body, generating structured descriptions. The "match-case" construct was also added as a new branch type to the "branch points" list in the prompt.

This also prompted us to reconsider the role of AST analysis. Attempting to exhaustively enumerate all syntax structures is impractical. A more sustainable strategy is to at least record the line numbers and types of unknown nodes, prompting the LLM to be aware of code blocks requiring attention. Additionally, AST analysis should complement source code text—the AST provides precise structural information, while source code offers complete semantic context.

2.3 The Context Window Trade-off: Why Providing Only Local Code Leads to Unreasonable Test Cases

In the closed-loop supplementation phase, to reduce prompt length and response latency, we initially sent only the uncovered line and its three surrounding lines of code to the LLM. This design was based on the notion that "local problems should be solved locally," but it exposed problems in practice.

For example, in one experiment, the uncovered line was return NumberWords.TEENS.value[ones] inside the convert_small_number function. The LLM received

only this line and its three surrounding lines of context, lacking the function signature, validity checks (if num < 0 and if num >= 100), and the calculation logic for tens, ones = divmod(num, 10). Consequently, the LLM generated convert_small_number(100) as input attempting to cover this branch. However, 100 is intercepted at the function entry by if num >= 100, raising an exception and never reaching the target branch.

The fundamental cause was that context window compression compromised logical integrity. For a branch within a function, the LLM requires the following information:

- Function signature (input type and value range)
- Preceding validity checks (which inputs are intercepted early)
- Dependency relationships for the current branch (the origin of tens and ones)

Providing only local code snippets prevents the LLM from acquiring this critical information. Therefore, we refactored the context extraction rules:

- If the uncovered line resides within a function, extract the entire function body (including signature, docstring, and all code lines)
- If the uncovered line is at the function level (e.g., an entire function never called), also extract the context where the function is called
- Explicitly state in the prompt: "Generated inputs must pass all validity checks at the function entry"
- Add code-level pre-filtering: discard inputs that would clearly trigger pre-exception conditions before concatenation

After refactoring, the proportion of unreasonable test cases decreased from approximately 40% to below 15%. Therefore, context compression must preserve logical integrity; for branches within functions, providing the complete function body is a prerequisite.

2.4 Conflict Between LLM "Common Sense" and Code "Facts": Logical Deviations in Floating Point and Rounding

In the Dayjs project, LLM-generated assertions exhibited two systematic error patterns:

- **Floating Point Precision**: For diff('2024-04-15', 'month', true), the LLM output expect(result).toBe(1.5), while the actual return value was 1.564516129032258.
- **Rounding Rules**: For diff('2024-03-01', 'month', false), the LLM expected 2 (rounding up), while the actual return value was 1 (flooring).

Both problems stem from the same underlying issue—the LLM relies on "common sense" rather than "code facts." Common sense suggests that from January 15 to March 1 is approximately 1.5 months, which rounds to 2. However, Dayjs's diff implementation is based on day count conversion, influenced by leap years and month lengths, and the integer mode uses Math.floor rather than rounding. The LLM learned general concepts of time differences from its training data but did not grasp the specific implementation details of Dayjs, leading to systematic biases.

To address this problem, we adopted a three-tier mitigation strategy:

- **Prompt Enhancement**: Added specific instructions in the script generation prompt: "Dayjs's diff method with float=true returns floating point values based on day count conversion, which may not be integers; with float=false, it returns floored integers, not rounded values."
- **Assertion Auto-conversion**: When the LLM generates toBe(1.5), automatically convert it to toBeCloseTo(1.5, 2), as strict equality for floating point numbers is almost always incorrect in testing.
- **Suspicious Assertion Logging**: Record warnings for assertions using strict equality with time differences for potential human review.

After these improvements, Dayjs project accuracy improved from 0% to 97.37%.

2.5 Uncontrollable Output Format: Multi-Layer JSON Parsing for Fault Tolerance

Despite repeated emphasis in prompts to "return only JSON, without any additional text," LLMs frequently add explanatory text, code block markers, or even output incomplete JSON. Typical error patterns:

- "Here are the test cases:\n```json\n[{...}]\n```"
- "I have generated the following test cases:\n[\n {...}\n]\n"
- "```\n[\n {...}\n]\n```" (without json identifier)
- "{...}\n{...}" (multiple individual objects, not an array)
- "[{\"id\":\"TC001\", ..." (truncated, unclosed brackets)

LLMs are trained on large corpora containing numerous examples formatted as "question +

explanation + answer." The models naturally gravitate toward conversational output patterns. Even when instructed to "return only JSON," the model may interpret "return only" as meaning "the final output should contain only JSON," treating preceding explanations as also part of the "output."

Therefore, we abandoned the expectation of complete reliance on the LLM adhering to format constraints and instead implemented a multi-layer parsing function that attempts parsing in sequence:

- Direct `json.loads(response)`
- Extract ```` ```json ... ``` ```` code blocks via regex
- Extract `[ ... ]` patterns without code block markers
- Extract `{ ... }` patterns (attempting to repair unclosed brackets for truncated cases)
- Scan line by line, extracting individual JSON objects and concatenating into an array

This function comprises approximately 100 lines of code but significantly improved parsing success rates. JSON parsing success rate increased from approximately 70% to near 100%.

2.6 Closed-loop Supplementation Stability: Rollback Mechanisms and Syntax Validation

During multi-round closed-loop supplementation, we encountered two stability-impacting issues:

- Coverage Regression: Round 1 coverage 71%, Round 2 86%, Round 3—the newly generated script had syntax errors causing Jest execution failure, coverage dropping to 0%.
- Syntax Errors: LLM-generated test fragments, after concatenation, frequently exhibited issues such as mismatched parentheses, unclosed quotes, or chain-call errors.

These two issues both point to a core contradiction—balancing the need to allow LLMs freedom to explore new paths for coverage improvement against the need to preserve existing achievements. In attempting to cover more code, LLMs may "overreach," compromising the integrity of existing scripts. Therefore, we constructed a "best version tracking + syntax validation" mechanism:

- Best Version Tracking: Record script content and coverage after each round. If new

coverage > historical best, update the best version; if new coverage < historical best, discard the current round's content and roll back to the best version.

- **Syntax Validation**: Immediately after concatenation, perform syntax checks (using node --check for JavaScript, ast.parse for Python). If syntax errors are detected, discard the supplement, retain the previous version, and log the error information.
- **Error Feedback**: If a supplement is discarded due to syntax errors, add a note to the next round's prompt: "The code generated in the previous round had syntax errors; please correct them."

After this, cases of "coverage dropping to zero after multiple supplementation rounds" no longer occurred. Script syntax correctness improved from 85% to nearly 100%.

2.7 Deduplication and Overfitting: Outstanding Issues Requiring Further Resolution

During multi-round closed-loop supplementation, the LLM may generate duplicate test cases. For instance, TC001 and TC042 both test convert_small_number(5), differing only in descriptive text. This increases test execution time without improving coverage. An effective deduplication mechanism has not yet been implemented, primarily because the LLM generates inputs in diverse forms (integers, floats, strings, etc.), making simple hash-based deduplication inadequate.

Additionally, despite prompt and context optimization, some experiments still exhibit cases where the LLM generates extreme inputs to cover a specific line of code. For example, to cover NumberWords.TENS.value[tens], the LLM generated num=200, which falls outside the valid range for convert_small_number, causing test failure. This essentially stems from incomplete understanding of preconditions or missing context for code fragments.

Therefore, for deduplication, introducing a deduplication mechanism based on input parameter hashing could be considered, but parameter normalization (e.g., treating 5 and 5.0 as equivalent) must first be addressed. For overfitting, adding hard constraints in the prompt stating that "generated test cases must pass all validity checks at function entry" could be planned, complemented by code-level pre-filtering.

3. Experimental Data Comparison

The following table presents comparative results before and after implementation of key

improvements:

Improvement item	project	metric	Before improvement	Improved	change
--script pattern	Dayjs	statement coverage	26.9%	72.7%	+45.8%
--script pattern	Dayjs	precision	0%	97.37%	+97.37%
AST assist	Python	Closed-loop cycle	3 rounds	Round 1	-67%
AST assist	Python	precision	69.23%	95.45%	+26.22%
Context optimization	Python	Supplementary use case pass rate	60%	85%+	+25%
Grammar check + rollback	be in common use	Script syntax accuracy rate	85%	99%+	+14%
Multi-level JSON parsing	be in common use	JSON parsing success rate	70%	99%+	+29%

4. Project Summary and Outlook

4.1 Core Project Outcomes

This project successfully constructed an LLM-based automatic white-box test case generation tool, realizing full-process automation spanning code analysis, test case generation, test execution, coverage measurement, and closed-loop supplementation. Key achievements are manifested in the following areas:

Complete Module Development: Seven core modules were built, including code analysis, LLM invocation, test case generation, coverage measurement, experiment tracking, and CLI entry point, realizing end-to-end automated white-box test case generation workflow with support for Python/JavaScript target languages and multiple LLM backends.

Key Technological Innovations:

1. **Adaptive Output Mode (--script)**: To address testing difficulties with JavaScript chain-call libraries, we designed both JSON test case and direct script generation modes, increasing Dayjs statement coverage from below 30% to above 97%.

2. **AST-integrated Analysis**: By precisely extracting code structure information through the Python AST module and injecting it into prompts, first-round test case quality improved significantly, reducing closed-loop rounds from 3 to 1.
3. **Multi-LLM Comparative Experimentation**: The --compare parameter enables automatic sequential execution across all configured LLM backends, generating comparison reports covering coverage, accuracy, and runtime metrics, providing data support for model selection.
4. **Stable Closed-loop Feedback Mechanism**: An automated "generation-execution-analysis-supplementation" closed-loop process was established. Through engineering approaches including fragment concatenation, rollback mechanisms, syntax validation, and multi-layer JSON parsing, issues such as LLM output truncation, coverage regression, and format instability were effectively resolved, resulting in final output scripts with syntax correctness exceeding 99% and JSON parsing success approaching 100%.
5. **Multi-dimensional Experimental Validation**: Dozens of experiments were conducted across two projects (convert_number_to_words.py and index.js), covering scenarios including presence/absence of requirement documentation, AST assistance, --script mode, and multi-model comparisons, with over 30 experimental data records collected. Experimental results validated the tool's effectiveness and revealed performance patterns of different LLMs across various code styles.

4.2 Project Experience Summary

Through the development and experimentation process of this project, we accumulated several practical experiences regarding AI-assisted software testing:

- **Abstraction Design Must Align with Code Style**: The JSON format is suitable for describing pure functional code but exposes expressiveness limitations when facing chain calls. When existing abstractions cannot adequately represent the target problem domain, reconstructing the abstraction is preferable to patching within the old framework. Having the LLM directly write test scripts essentially acknowledges that "code itself is the most precise test description language."
- **Context Integrity Takes Priority Over Length Optimization**: While compressing context during closed-loop supplementation can reduce prompt length and response latency,

compromising logical integrity prevents the LLM from correctly understanding code reachability. For branches within functions, providing the complete function body is a prerequisite; for cross-function dependencies, the call chain context must also be supplied.

- **Engineering Fault Tolerance Is More Reliable Than Format Constraints**: Regardless of how strongly prompts emphasize "return only JSON," LLMs may still add explanations or output malformed formats. Rather than attempting to alter model behavior, constructing multi-layer parsing and fault tolerance mechanisms at the code level is more effective. The four-layer JSON parsing, syntax validation, and rollback mechanisms in this project exemplify this philosophy.
- **Preserving the Best Historical Version Is Essential**: LLMs exploring new paths may compromise existing achievements; thus, maintaining the best version state is necessary. Rollback mechanisms ensure that even if a particular round's generation quality declines, the final output does not regress.
- **AST Analysis Requires Continuous Evolution**: Language syntax continually evolves (e.g., Python's match-case), and analyzers must expand accordingly; otherwise, structural information omissions will occur. A more sustainable strategy is to design generic node handling logic that records at least location information for unknown nodes for LLM reference.

4.3 Future Development Directions

Based on the achievements and limitations of this project, combined with trends in LLM technology evolution, future optimization and expansion can proceed in the following directions:

- **Enhance AST Analysis Capabilities**: Extend the Python analyzer to support additional syntax structures including async/with, try-except-else, and more; implement JavaScript AST analysis modules to enable structured information injection for JS projects; explore AST-based path constraint generation to complement LLM capabilities.
- **Integrate Traditional Testing Techniques**: Combine LLMs with symbolic execution and fuzzing. Leverage symbolic execution to solve extreme boundary constraints and generate test cases covering complex conditions; leverage LLMs to generate test cases covering foundational paths and business rules, achieving complementary strengths and enhancing testing comprehensiveness.

- **Expand Multi-language and Framework Support**: Add support for mainstream programming languages including Java, Go, and C++, along with adaptation to common testing frameworks such as JUnit, TestNG, and Google Test, improving the tool's general applicability.
- **Implement Parallelization and Performance Optimization**: Introduce multi-threaded/distributed execution mechanisms to parallelize calls to multiple LLM backends and test case execution, significantly reducing total runtime. Simultaneously implement test case deduplication (based on input parameter hashing) to reduce redundancy.
- **Enhance Test Case Validity Verification**: Develop LLM-based test case validation modules to automatically detect whether generated test cases conform to code logic and business rules, filtering invalid or erroneous test cases. This can be combined with static analysis results to perform pre-judgment on anomalous inputs.
- **Continuously Optimize Prompt Engineering**: Based on LLM limitations discovered during experimentation, design more refined prompt templates. Introduce few-shot learning by including representative test case examples in prompts; introduce chain-of-thought (CoT) prompting to guide LLMs through step-by-step reasoning of code logic and test case design, reducing hallucinations.

4.4 Overall Project Conclusion

This project successfully demonstrated the feasibility and effectiveness of white-box test case automatic generation technology based on large language models. By integrating AST analysis, requirement documentation, and closed-loop feedback mechanisms with LLMs, the tool achieved performance comparable to manual white-box testing on pure functional code (91% statement coverage, 92.3% branch coverage), and achieved breakthrough improvements on chain-call libraries (97% accuracy). The LLM limitations exposed during experimentation — including insufficient context understanding, unstable output formats, and logical biases—were effectively mitigated through engineering approaches such as fragment concatenation, rollback mechanisms, multi-layer parsing, and syntax validation.

The core insight of this project is that the focus of AI testing tool development should not be on pursuing perfect one-shot outputs from LLMs, but on constructing engineering systems capable

of tolerating imperfect outputs. As LLM technology continues to advance, particularly with improvements in code understanding and reasoning capabilities, LLM-based white-box test automatic generation tools will play increasingly important roles in software testing. The future deep integration of traditional testing techniques with AI technologies is poised to drive software testing toward higher levels of automation and intelligence.